

Phương pháp tối ưu hóa cho một lớp bài toán quy hoạch động

Lin Tao

Nguồn gốc: Optimization Methods for a Class of Dynamic Programming Problems

Dynamic Programming / Queue-optimized DP - Lin Tao.pdf

Tóm tắt

Hiện nay các bài toán quy hoạch động đã xuất hiện rất thường xuyên. Phương trình chuyển trạng thái thường không khó để lập ra, nhưng phần tối ưu thuật toán lại hay gây trở ngại. Bài viết này thông qua một số bài toán điển hình trong vài năm gần đây để thảo luận một phương pháp tối ưu cho một lớp bài toán quy hoạch động. Các tối ưu trong những bài này ít nhiều đều lấy cảm hứng từ bài *Batch* của IOI 2002; trước khi đọc, có thể tham khảo báo cáo lời giải của Yu Wei.

1 Mở đầu

Có một lớp bài toán quy hoạch động mà biến quyết định có một số tính đơn điệu nhất định. Dựa vào đặc điểm này, ta có thể tối ưu việc chọn biến quyết định, từ đó giảm độ phức tạp thời gian của thuật toán. Các ví dụ sau minh họa cách áp dụng tư tưởng đó.

2 Euro (Balkan 2003)

Đề bài

Cho một dãy $a_1, a_2, \dots, a_n$ và một số T . Cần chia dãy thành một số đoạn. Mỗi đoạn có một trọng số F . Nếu chia $a_i, \dots, a_j$ thành một đoạn thì trọng số đoạn này được tính bởi

$$F = \text{Sum}[i, j] \cdot j - T, \quad \text{Sum}[i, j] = \sum_{k=i}^j a_k.$$

Hãy tìm một cách chia sao cho tổng trọng số của các đoạn là lớn nhất.

Ràng buộc:

$$1 \leq n \leq 34567, \quad -1000 \leq a_i \leq 1000 \quad (i = 1, 2, \dots, n).$$

Phân tích

Rất dễ nghĩ đến quy hoạch động. Gọi $F[i]$ là tổng trọng số tối ưu khi chia i phần tử đầu của dãy, và $t[i]$ là tổng của i phần tử đầu. Khi đó phương trình chuyển trạng thái là

$$F[i] = \max_{0 \leq k < i} \{F[k] + (t[i] - t[k]) \cdot i - T\}, \quad 0 < i \leq n.$$

Điều kiện biên là $F[0] = 0$. Độ phức tạp thời gian của thuật toán trực tiếp là $O(n^2)$. Liệu có thể tối ưu thêm hay không?

Trước hết, ta phân tích cấu trúc của một phương án chia tối ưu. Giả sử có một đoạn $a_i, \dots, a_j$ với $j < n$. Nếu tồn tại $i \leq k \leq j$ sao cho tổng của $a_i, \dots, a_{k-1}$ là âm, còn tổng của $a_k, \dots, a_j$ là dương, thì cách chia này chắc chắn không tối ưu. Lý do là ta có thể giữ nguyên các đoạn khác, tách riêng $a_i, \dots, a_{k-1}$, đồng thời gộp $a_k, \dots, a_j$ vào đoạn kế tiếp. Khi đó phần có tổng âm được nhân với một chỉ số nhỏ hơn, còn phần có tổng dương được nhân với một chỉ số lớn hơn, nên tổng trọng số tăng lên.

Vì vậy ta có kết luận sau: nếu $a_i, \dots, a_j$ với $j < n$ là một đoạn con trong phương án chia tối ưu, thì nhất định phải có

$$t[j] \leq t[i - 1].$$

Nếu không, ta sẽ tìm được một vị trí k như trên để cải thiện phương án.

Tiếp theo, xét biến quyết định k của $F[i]$. Đặt

$$S[k, i] = F[k] + (t[i] - t[k]) \cdot i - T.$$

Với mọi $0 \leq k_1 < k_2 < i$, ta có

$$S[k_1, i] = F[k_1] + (t[i] - t[k_1]) \cdot i - T,$$

$$S[k_2, i] = F[k_2] + (t[i] - t[k_2]) \cdot i - T,$$

nên

$$S[k_1, i] - S[k_2, i] = F[k_1] - F[k_2] + (t[k_2] - t[k_1]) \cdot i.$$

Trường hợp 1: $t[k_2] \geq t[k_1]$. Khi đó tổng của $a_{k_1+1}, \dots, a_{k_2}$ là dương. Do đó, dù dãy $a_1, \dots, a_{k_2}$ được chia như thế nào, trong phương án tương ứng tất yếu xảy ra một trong hai tình huống: có một đoạn có tổng dương, hoặc phần đuôi của một đoạn có tổng dương. Cả hai tình huống đều cho thấy đây không phải phương án tối ưu. Nói cách khác, k_2 không có ý nghĩa làm quyết định.

Trường hợp 2: $t[k_2] < t[k_1]$. Nếu

$$S[k_1, i] - S[k_2, i] = F[k_1] - F[k_2] + (t[k_2] - t[k_1]) \cdot i > 0,$$

tức là

$$i < \frac{F[k_2] - F[k_1]}{t[k_1] - t[k_2]},$$

thì với mọi $j \leq i$ ta cũng có

$$S[k_1, j] - S[k_2, j] > 0.$$

Như vậy biến quyết định có tính đơn điệu.

Đặt

$$g(j, k) = \frac{F[k] - F[j]}{t[j] - t[k]}.$$

Với $0 < k_1 < k_2 < k_3 < \dots < k_m$, ta có thể duy trì một hàng đợi sao cho

$$t[k_1] > t[k_2] > t[k_3] > \dots > t[k_m],$$

đồng thời bảo đảm

$$i < g(k_2, k_1) < g(k_3, k_2) < g(k_4, k_3) < \dots < g(k_m, k_{m-1}).$$

Khi đó

$$S[k_1, i] > S[k_2, i] > S[k_3, i] > \dots > S[k_m, i],$$

và quyết định tối ưu của $F[i]$ là k_1 :

$$F[i] = F[k_1] + (t[i] - t[k_1]) \cdot i - T.$$

Cài đặt

Các thao tác duy trì hàng đợi như sau.

1. Thêm một phần tử mới k . Vì $k > k_m$, ta đặt k vào cuối hàng đợi. Nếu

$$g(k, k_m) < g(k_m, k_{m-1}),$$

thì xóa k_m , tức $m \leftarrow m - 1$, và lặp lại cho đến khi

$$g(k_m, k_{m-1}) < g(k, k_m).$$

2. Xóa phần tử đầu. Nếu

$$g(k_2, k_1) < i,$$

thì xóa k_1 và lặp lại cho đến khi

$$g(k_2, k_1) > i.$$

Vì tổng số phần tử được thêm và bị xóa đều không vượt quá n , độ phức tạp thời gian của thuật toán là $O(n)$.

3 Giá trị trung bình lớn nhất (USACO)

Đề bài

Cho một dãy $a_1, a_2, \dots, a_n$. Hãy tìm một đoạn có độ dài ít nhất là L sao cho giá trị trung bình của đoạn đó là lớn nhất.

Phân tích

Giả sử trong các đoạn kết thúc tại a_i , đoạn có giá trị trung bình lớn nhất bắt đầu từ phần tử thứ $F[i]$. Khi đã biết $F[i]$, ta xét cách tìm $F[i + 1]$.

Giả sử $F[i + 1] < F[i]$. Điều đó có nghĩa là giá trị trung bình của đoạn từ $F[i + 1]$ đến $F[i] - 1$ lớn hơn giá trị trung bình của đoạn từ $F[i]$ đến $i + 1$. Tuy nhiên, theo định nghĩa, giá trị trung bình của đoạn từ $F[i + 1]$ đến $F[i] - 1$ không thể lớn hơn giá trị trung bình của đoạn từ $F[i]$ đến i . Vì vậy đoạn tốt nhất kết thúc tại $i + 1$ sẽ không vượt qua đoạn tốt nhất kết thúc tại i , nên trường hợp này không có ý nghĩa.

Từ kết luận trên, khi tính giá trị trung bình lớn nhất, ta chỉ cần quan tâm tới các đoạn kết thúc tại $i + 1$ trong trường hợp $F[i + 1] \geq F[i]$.

Giả sử ta đã tính được $F[i]$. Trong quá trình tính các trạng thái về sau, vì vị trí bắt đầu của đoạn liên tục tăng, ta cần xét khi nào nên tăng và tăng bao nhiêu. Gọi giá trị trung bình lớn nhất hiện tại là x , vị trí bắt đầu hiện tại là p , và vị trí cuối là i . Ta xét đoạn các vị trí có thể xóa từ p đến $i - L$. Nếu giá trị trung bình từ p đến một vị trí nào đó nhỏ hơn giá trị trung bình lớn nhất hiện tại, thì phần đó không còn ý nghĩa và có thể xóa trực tiếp.

Giả sử giá trị trung bình của các đoạn

$$p \dots p, \quad p \dots p + 1, \quad p \dots p + 2, \dots$$

lần lượt là $x_0, x_1, x_2, \dots$. Nếu $x_i > x_{i+1}$, thì sau khi xóa đoạn p đến $p + i$, chắc chắn ta cũng sẽ xóa tiếp $p + i + 1$. Sau khi xóa $p + i + 1$, việc có nên xóa $p + i + 2$ hay không lại cần được tính lại.

Nhưng việc tính lại này là không cần thiết, vì ta có thể tránh trước. Nguyên nhân phải tính lại là trong dãy xuất hiện hiện tượng $x_i > x_{i+1}$. Do đó ta duy trì một dãy vị trí

$$P_1, P_2, P_3, \dots,$$

trong đó x_i là giá trị trung bình của đoạn từ P_i đến $P_{i+1} - 1$, và các giá trị này thỏa

$$x_1 < x_2 < x_3 < x_4 < \dots.$$

Vì phần có thể xóa không được vượt quá $i - L$, nên khi i tăng, phần có thể xóa cũng tăng. Mỗi khi dãy này được mở rộng, ta phải duy trì tính chất của dãy P .

Giả sử ban đầu ta có $P_1, P_2, P_3, \dots, P_m$, và vị trí mới được phép xóa tăng đến $P_{m+1} - 1$. Nếu $x_m > x_{m-1}$, tính chất của dãy không đổi nên không cần xử lý. Nếu $x_m < x_{m-1}$, ta gộp đoạn từ P_m đến $P_{m+1} - 1$ vào đoạn trước đó, xóa P_m , rồi lặp lại thao tác này.

Bản chất của thao tác trên là: nếu giá trị trung bình của đoạn cuối nhỏ hơn giá trị trung bình của đoạn áp chót, ta gộp hai đoạn đó thành một đoạn. Vì mỗi phần tử trong dãy chỉ được thêm hoặc xóa một lần, độ phức tạp của thuật toán là $O(n)$.

Bạn đọc cũng có thể tham khảo phương pháp kết hợp hình học và đại số mà Zhou Yuan từng nhắc đến ở trại mùa đông để giải bài này.

4 Máy tính kiểu mới (OIBH)

Đề bài

Cho một dãy số. Ta cần thay đổi dãy sao cho nó thỏa điều kiện sau: bắt đầu từ số đầu tiên, đọc vào một số i , sau đó đọc ra đúng i số đứng ngay sau nó; tiếp tục đọc vào một số j , rồi đọc ra đúng j số đứng ngay sau nó; và cứ lặp lại như vậy, sao cho toàn bộ dãy vừa được đọc hết đúng lúc. Chi phí thay đổi dãy là tổng các giá trị tuyệt đối của chênh lệch trước và sau khi thay đổi trên mọi phần tử. Hãy tìm một phương án có chi phí thay đổi nhỏ nhất.

Phân tích

Khi đọc đề lần đầu, ta rất dễ nghĩ ra phương trình quy hoạch động. Gọi $F[i]$ là chi phí nhỏ nhất để đoạn bắt đầu tại vị trí i được đọc hết đúng cách. Khi đó

$$F[i] = \min_{k > i} \{F[k] + |a_i - (k - i - 1)|\}.$$

Độ phức tạp của thuật toán này hiển nhiên là $O(n^2)$.

Đặt

$$S[i, k] = F[k] + |a_i + i + 1 - k|.$$

Nếu với $k < j$ có

$$S[i, k] < S[i, j],$$

thì

$$|a_i + i + 1 - k| - |a_i + i + 1 - j| < F[j] - F[k].$$

Tập nghiệm của bất đẳng thức này có thể có dạng

$$a_i + i + 1 < x.$$

Dựa vào tính đơn điệu này, ta thử xây dựng một dãy

$$k_1 > k_2 > k_3 > \dots$$

sao cho tập nghiệm của bất đẳng thức

$$S[i, k_{r+1}] < S[i, k_r]$$

có dạng

$$a_i + i + 1 < x_r,$$

và đồng thời

$$x_1 > x_2 > x_3 > x_4 > \dots.$$

Khi đã có một dãy như vậy, để tính $F[i]$ ta không cần liệt kê mọi biến quyết định k . Thay vào đó, ta chỉ cần tìm chỉ số lớn nhất r sao cho

$$a_i + i + 1 < x_r,$$

rồi lấy k_r làm quyết định.

Dĩ nhiên, dãy $k_1, k_2, k_3, \dots$ được tăng dần trong quá trình xử lý. Khi thêm một phần tử mới k_{m+1} , ta giải bất đẳng thức

$$S[i, k_{m+1}] < S[i, k_m]$$

và thu được tập nghiệm $a_i + i + 1 < x_{m+1}$. Nếu $x_{m+1} < x_m$ thì tính chất của dãy không đổi; ngược lại, ta xóa k_m và lặp lại thao tác này cho đến khi dãy lại thỏa

$$x_1 > x_2 > x_3 > x_4 > \dots.$$

Độ phức tạp của các thao tác duy trì là $O(n)$, nên độ phức tạp toàn bài là $O(n \log n)$.

Thực ra bài này còn có một cách giải tốt hơn bằng đồ thị, có thể giảm độ phức tạp xuống $O(n)$.

5 Tổng kết

Với các bài toán trên, việc viết phương trình quy hoạch động không khó. Tuy nhiên, theo cách thông thường, biến quyết định cần được liệt kê. Bài viết này tập trung vào một số tính đơn điệu của biến quyết định, rồi tối ưu thêm ở khâu chọn quyết định, từ đó đạt mục tiêu giảm độ phức tạp thời gian.

Các lời giải trên có một số điểm chung, cũng là các điểm chung của kiểu tối ưu này:

1. Từ phương trình quy hoạch động, phát hiện một số tính đơn điệu của biến quyết định.
2. Dùng tính đơn điệu đó để duy trì một hàng đợi thể hiện chính tính chất đơn điệu.
3. Dùng hàng đợi này để chọn quyết định.